MiniGestor de Base de Datos Multimodal

Informe incremental — Partes 1 y 2
(base de datos relacional y base de datos espacial)

Universidad de Ingeniería y Tecnología
Curso: Base de Datos 2 — Ciclo 2026-2

Integrante	**Código**
Sebastian Loli Gonzalez	202420022
Tamy Kristel Flores Guevara	202310472
Fabricio Bautista	202310283
Diego Roger Illescas Castillo	202310417
Badi Masud Rodriguez Ramirez	202310299

Octubre de 2026

Índice

1. Introducción **3**
1.1. Repositorio . . . 3

2. Diseño arquitectónico **3**
2.1. Vista general por capas . . . 3
2.2. Flujo de una consulta . . . 4
2.3. Organización del código fuente . . . 5
2.4. Decisiones de diseño relevantes . . . 5

3. Dominio de datos **6**
3.1. Tipos y representación de registros . . . 6
3.2. Tablas de demostración . . . 6
3.3. Tablas y datos definidos por el usuario . . . 6
3.4. Conjuntos de datos de los experimentos . . . 6

4. Parte 1: algoritmos y estructuras **7**
4.1. Heap File . . . 7
4.2. Archivo Secuencial Paginado . . . 7
4.3. Árbol B+ . . . 7
4.4. Hash Extendible . . . 8
4.5. Algoritmos externos . . . 8
4.6. Parser SQL y planner . . . 8
4.7. Transacciones y concurrencia . . . 9

4.8. Interfaz de usuario 10

5. Parte 2: base de datos espacial **10**

5.1. Alcance 10
5.2. Estructura del R-Tree 10
5.3. Inserción y división cuadrática 11
5.4. Carga masiva 11
5.5. Consulta por rango 11
5.6. k vecinos más cercanos (best-first) 11
5.7. Métricas de distancia y plano del índice 12
5.8. Intersección con polígonos 12
5.9. Integración con el motor 12
5.10. Extensión del lenguaje SQL 12
5.11. Visualización en mapa 13

6. Evaluación experimental de la Parte 1 **13**

6.1. Metodología 13
6.2. Heap File vs Archivo Secuencial Paginado 14
6.3. B+ agrupado vs B+ no agrupado vs Hash Extendible 16
6.4. Resumen de resultados de la Parte 1 19

7. Evaluación experimental de la Parte 2 **20**

7.1. Metodología 20
7.2. Consultas por rango 20
7.3. k vecinos más cercanos 22
7.4. Construcción del índice y espacio 23
7.5. Intersección con polígonos 24
7.6. Interpretación de resultados 25
7.7. Comparación cualitativa 26

8. Pruebas y verificación **26**

9. Limitaciones y trabajo futuro **27**

10. Conclusiones **27**

A. Cómo reproducir los experimentos **28**

1. Introducción

Este informe documenta el diseño y la implementación incremental de un gestor de bases de datos que se extenderá progresivamente para soportar datos relacionales, espaciales, textuales y multimedia. El presente avance se concentra en las dos primeras partes del proyecto:

- **Parte 1 — Base de datos relacional:** almacenamiento en disco (Heap File y Archivo Secuencial Paginado), indexación (B+ agrupado, B+ no agrupado y Hash Extendible), algoritmos externos (*External Sort* y *External Hashing*), parser y optimizador SQL, transacciones con control de concurrencia, interfaz gráfica de cuatro paneles y comparación experimental.
- **Parte 2 — Base de datos espacial:** índice R-Tree para puntos geográficos, consultas por rango, k vecinos más cercanos e intersección con polígonos, con distancias Euclidiana y Geodésica (Haversine), extensión del lenguaje SQL, visualización en mapa y comparación experimental.

Las Partes 3 a 5 (texto, multimedia y aplicación con inteligencia artificial) quedan fuera del alcance de este avance. De acuerdo con el enunciado, el informe incluye el diseño arquitectónico, el dominio de datos, la explicación de los algoritmos y la parte experimental.

1.1. Repositorio

El código fuente y los archivos necesarios para reproducir los experimentos se encuentran en el repositorio del proyecto:

https://github.com/tamitooo/Proyecto-BD2

2. Diseño arquitectónico

2.1. Vista general por capas

El sistema se divide en tres capas: una interfaz web, una API REST y el motor de base de datos, implementado en Python con la biblioteca estándar. La interfaz nunca conoce detalles físicos (páginas, RIDs, estructuras de índice): solo intercambia JSON con la API.

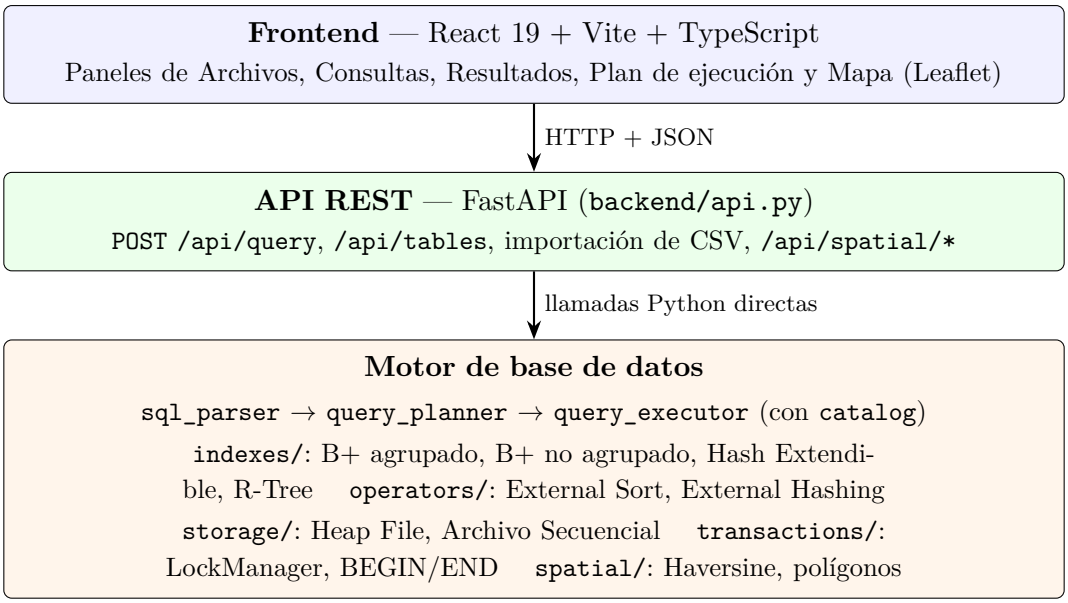


Figura 1: Arquitectura por capas del MiniGestor.

2.2. Flujo de una consulta

1. El usuario escribe SQL en el panel de consultas y el frontend envía `POST /api/query` con `{"sql": "..."}`.
2. `backend/engine.py` entrega el texto al `QueryExecutor`.
3. El **parser** tokeniza y valida la sentencia y construye el árbol sintáctico (`SelectStatement`, `InsertStatement`, `DeleteStatement`, DDL, etc.) con un `QuerySpec` (tabla, predicados, `GROUP BY`, `ORDER BY`, joins).
4. El **planner** elige la ruta de acceso comparando candidatos (Sección 4.6) y emite un plan en el que cada paso lleva su justificación (`reason`) y sus detalles (`details`).
5. El **ejecutor** resuelve la tabla en el `Catalog`, lee del almacenamiento físico, aplica índices y operadores externos y devuelve un `QueryResult`.
6. El resultado se serializa a JSON y React muestra los datos y el plan, con una traza de ejecución real (pasos y tiempos).

2.3. Organización del código fuente

Carpeta	Contenido
`storage/`	`record.py` (esquema y serialización binaria), `heap_file.py`, `sequential_file.py`
`indexes/`	`bplus_tree.py`, `clustered_bplus.py`, `unclustered_bplus.py`, `extendible_hash.py`, `rtree.py`
`spatial/`	`geo.py` (distancias, polígonos, generador de datos), `index.py` (`SpatialIndex`: R-Tree con latitud y longitud)
`operators/`	`external_sort.py`, `external_hashing.py`
`query/`	`sql_parser.py`, `query_planner.py`, `catalog.py`, `query_executor.py`, `table_manager.py`, `spatial_queries.py`, `csv_loader.py`, `query_result.py`
`transactions/`	`lock_manager.py`, `transaction_manager.py`, `demo_concurrencia.py`
`backend/`	`api.py` (endpoints FastAPI), `engine.py` (catálogo de demostración)
`frontend/`	aplicación React: `src/panels/` (un componente por panel) y cliente HTTP
`tools/`	`import_csv.py`: carga de CSV por línea de comandos
`benchmarks/`	scripts de medición y generadores de gráficas; resultados en `benchmark_results/`
`tests/`	pruebas unitarias, de integración y *end-to-end*

Tabla 1: Organización del repositorio.

2.4. Decisiones de diseño relevantes

- **Contrato de respuesta estable.** `QueryResult` expone `success`, `columns`, `rows`, `affected_rows`, `execution_time_ms`, `execution_plan` y `error`. El mismo contrato serviría para una API GraphQL futura.
- **Planner basado en reglas y explicable.** Cada paso del plan indica por qué se eligió; el plan que se muestra es el que realmente se ejecuta.
- **Reorganización e índices.** Reorganizar el Archivo Secuencial reescribe `.main` completo, por lo que *todos los RID cambian*; el ejecutor reconstruye los índices cuando una inserción dispara una reorganización.
- **Persistencia de índices.** La definición de cada índice queda en el manifiesto del catálogo y el índice se reconstruye (por carga masiva) al abrirlo.
- **Acceso concurrente en la API.** El API atiende peticiones en un *threadpool*; el motor toma un `threading.Lock` antes de ejecutar cada sentencia para no corromper archivos desde dos peticiones simultáneas.
- **Un solo ejecutor.** `query/query_executor.py` es el *único* ejecutor del motor: lo usan el API, la interfaz, las demos y todas las pruebas. El ejecutor anterior (`query/executor.py`, PR #37) se eliminó y sus pruebas se migraron para que no existan dos semánticas distintas.
- **Tablas dinámicas por dos puertas.** El formulario del panel de archivos (`POST /api/tables`) construye el mismo `CREATE TABLE` que el usuario escribiría a mano, así que el índice Hash de la clave primaria, el registro en el catálogo y la persistencia se comportan igual por las dos vías.

3. Dominio de datos

3.1. Tipos y representación de registros

El esquema soporta tres tipos: `INT`, `FLOAT` y `VARCHAR(n)`. Cada tabla declara su clave primaria. `Schema` y `Field` (`storage/record.py`) serializan cada registro a un formato binario de **tamaño fijo** con el módulo `struct`, más un byte de estado (`EMPTY`, `USED` o `DELETED`). El tamaño fijo permite calcular la posición de cualquier registro a partir de su número de slot, y es la base tanto del Heap File como del Archivo Secuencial.

3.2. Tablas de demostración

La interfaz trabaja sobre tres tablas registradas al iniciar la API, elegidas para que se vean las dos técnicas de almacenamiento y las tres de indexación:

Tabla	Almacenamiento	Columnas	Índices
`users`	Heap File	`id INT` (PK), `name`, `age`, `dept`	Hash Extendible (único) sobre `id`
`employees`	Archivo Secuencial	`id INT` (PK), `name`, `dept`, `salary FLOAT`	B+ agrupado (`salary`); B+ no agrupado (`dept`)
`departments`	Heap File	`id INT` (PK), `name`, `city`	—

Tabla 2: Modelo de datos de demostración.

3.3. Tablas y datos definidos por el usuario

El usuario puede crear cualquier tabla con `CREATE TABLE` (nombre, columnas y tipos a elección) y cargar datos desde un CSV. Hay dos puertas de creación y todas desembocan en la misma sentencia SQL: (i) escribir el `CREATE TABLE` directamente en el editor SQL; (ii) usar el formulario del panel de archivos, que construye la sentencia y la ejecuta por el mismo camino. Gracias a esto, el índice Hash de la clave primaria, la persistencia en `catalog.json` y la reconstrucción al arrancar se comportan igual por las dos vías.

El cargador de CSV (`query/csv_loader.py`) inserta por el mismo camino que `INSERT`, admite campos entre comillas con comas internas y tildes, permite elegir el **separador** (coma, punto y coma de Excel en español, o tabulador) y si la primera fila es el **encabezado**. Las columnas se mapean *por nombre de encabezado* y, si el encabezado no cubre las columnas de la tabla, la importación falla con un mensaje que lista las columnas esperadas en lugar de corromper datos en silencio (el mapeo por posición solo se activa si el usuario indica `has_header=False`). El motor no infiere el esquema a partir del CSV: la tabla se declara antes con `CREATE TABLE`.

3.4. Conjuntos de datos de los experimentos

- **Parte 1:** registros `(id INT, nombre VARCHAR(30), nota FLOAT)` de 43 bytes, con semilla fija 42 (mismo conjunto y mismo orden de inserción para todas las técnicas), en tamaños de 1 000, 10 000 y 100 000 registros (50 000 en la comparación de índices).
- **Parte 2:** puntos sintéticos `(latitud, longitud)` con semilla 42, generados alrededor de un centro con focos de densidad, de modo que los rectángulos mínimos (MBR) del R-Tree se solapen, que es el caso interesante para el índice.

4. Parte 1: algoritmos y estructuras

4.1. Heap File

El Heap File (`storage/heap_file.py`) divide el archivo en **páginas de 4 096 bytes**. Cada página contiene $\lfloor 4096/\text{tamaño del registro} \rfloor$ slots (95 slots para registros de 43 bytes); los bytes sobrantes de la página no se usan. Un registro se identifica con su `RID = (página, slot)`.

- **Inserción:** se escribe en el primer slot libre que indica la *free-list* (persistida en un archivo `.free` con las páginas que tienen huecos); si no hay espacio libre, se agrega una página al final. Costo $O(1)$ amortizado.
- **Borrado:** marca el slot como `DELETED` (*tombstone*) y anota la página en la *free-list*; el espacio se reutiliza en línea en la siguiente inserción, sin compactar.
- **Búsqueda:** no existe un índice de clave primaria, por lo que cualquier consulta selectiva requiere un *full scan* $O(N)$.

4.2. Archivo Secuencial Paginado

El Archivo Secuencial (`storage/sequential_file.py`) mantiene los registros ordenados por clave primaria en dos regiones: el archivo principal `.main`, siempre ordenado, y un área de desbordamiento `.aux` donde se agregan las inserciones nuevas.

- **Búsqueda por clave:** búsqueda binaria sobre `.main` ($O(\log N)$) más un barrido del `.aux`.
- **Inserción:** se agrega al final de `.aux`, de modo que el orden se mantiene lógicamente sin reescribir el archivo en cada operación.
- **Eliminación lazy:** el registro queda marcado (*tombstone*) y se contabiliza como espacio desperdiciado.
- **Reorganización:** cuando el espacio desperdiciado supera el **30 %**, se reescribe `.main` fusionando `.main` y `.aux`, ordenado y compactado, y se vacía el `.aux`. Es una operación $O(N)$.
- **Consecuencia:** la reorganización cambia la posición de todos los registros, así que los índices existentes se reconstruyen (`Catalog.rebuild_indexes()`).

4.3. Árbol B+

`BPlusTree` implementa un árbol B+ de orden configurable con hojas enlazadas, *split* de hojas y nodos internos al desbordar, y eliminación con rebalanceo (préstamo o fusión de nodos hermanos). Soporta búsqueda puntual, `range_search(inicio, fin)` recorriendo las hojas enlazadas, recorrido ordenado completo y un método `validate()` que comprueba las invariantes. La búsqueda cuesta $O(\log_B N)$ y un rango $O(\log_B N + k)$, con k el número de resultados. Sobre esta base hay dos variantes:

- **B+ agrupado** (`ClusteredBPlusIndex`): las hojas almacenan el **registro completo**; por tanto, una búsqueda por clave o por rango devuelve los datos sin acceder al archivo de datos.
- **B+ no agrupado** (`UnclusteredBPlusIndex`): las hojas almacenan pares `(clave, RID)`, de modo que cada fila recuperada exige una lectura adicional por RID en el almacenamiento. Admite claves repetidas y la opción de unicidad.

4.4. Hash Extendible

`ExtendibleHash` mantiene un **directorio** de 2^d punteros, con d la *profundidad global*, que apunta a buckets de capacidad fija; cada bucket tiene su propia *profundidad local*. La búsqueda calcula el hash de la clave, usa los d bits indicados para indexar el directorio y examina un único bucket: $O(1)$ promedio.

1. Si el bucket destino tiene espacio, se inserta.
2. Si está lleno y su profundidad local es menor que la global, se divide el bucket en dos y se redistribuyen las claves según el siguiente bit del hash.
3. Si está lleno y su profundidad local iguala a la global, primero se **duplica el directorio** y luego se divide el bucket.
4. En el borrado, buckets vacíos o hermanos se fusionan y el directorio se reduce cuando es posible.

El crecimiento es dinámico y sin rebalanceo de árbol, pero el hash **no preserva el orden**: no sirve para rangos ni para `ORDER BY`.

4.5. Algoritmos externos

External Sort (`ORDER BY`). Se leen los registros en bloques que caben en memoria (límite configurable en registros); cada bloque se ordena y se escribe como un *run* ordenado en un archivo temporal. Luego se fusionan los runs con un **merge de k vías**, leyendo un registro por run, y se eliminan los archivos temporales. Costo $O(N \log N)$ con memoria acotada. Se usa cuando el `ORDER BY` no puede resolverse con un índice B+.

External Hashing (`GROUP BY` y `JOIN`). Los registros se reparten en particiones en disco mediante una función hash sobre la clave de agrupación o de unión, y cada partición se procesa por separado en memoria. Para `GROUP BY` se calculan los agregados (`count`, `sum`, `avg`, `min`, `max`) por partición. Para el `JOIN` (*Grace hash join*) ambas relaciones se particionan con la misma función hash y solo se combinan las particiones correspondientes. Costo $O(N + M)$ promedio, solo para equi-joins; para condiciones no de igualdad el planner recurre a bucle anidado.

4.6. Parser SQL y planner

El parser (`query/sql_parser.py`) soporta `SELECT` (con `WHERE`, `ORDER BY`, `GROUP BY`, `LIMIT`, agregados y joins), `INSERT`, `UPDATE`, `DELETE`, `CREATE/DROP TABLE`, `CREATE/DROP INDEX`, `BEGIN/END TRANSACTION`, `COMMIT`, `ROLLBACK` y `EXPLAIN [ANALYZE]`. Ignora comentarios (`-` y `/* */`) respetando los literales de texto, resuelve los nombres de columna sin distinguir mayúsculas y acepta scripts con varios `;` de nivel superior (toma la primera sentencia no vacía). El planner es un optimizador **basado en reglas**. Sus decisiones sobre la ruta de acceso son:

Condición	Ruta de acceso elegida
Igualdad sobre columna con Hash	`HASH_INDEX_LOOKUP`
Igualdad sobre columna con B+	`BPLUS_CLUSTERED_LOOKUP` / `BPLUS_UNCLUSTERED_LOOKUP`
Rango sobre columna con B+ (ambos extremos en un solo `range_search`)	`BPLUS_*_RANGE_SCAN`
`ORDER BY` ascendente sobre la columna del B+	`BPLUS_*_INDEX_SCAN` (evita el External Sort)
`OR` de igualdades sobre la misma columna indexada	`HASH_INDEX_UNION` / `BPLUS_INDEX_UNION`
Cualquier otro caso	`HEAP_SCAN` o `SEQUENTIAL_SCAN` más `FILTER`

Tabla 3: Reglas de elección de ruta de acceso.

La unión de índices para el `OR` solo se aplica cuando *todos* los grupos son igualdades sobre la misma columna indexada; en otro caso se perderían filas del otro grupo y el planner cae a escaneo con filtro. El plan es honesto: si no existe índice, el plan dice `HEAP_SCAN` y la lista de índices usados está vacía. Como ejemplo, `CREATE INDEX` cambia el plan en vivo:

```
EXPLAIN SELECT * FROM alumnos WHERE nota >= 14 ORDER BY id;
  -- HEAP_SCAN (no hay índice sobre nota)
CREATE INDEX idx_alumnos_nota ON alumnos (nota);
EXPLAIN ANALYZE SELECT * FROM alumnos WHERE nota >= 14 ORDER BY id;
  -- BPLUS_UNCLUSTERED_RANGE_SCAN usando idx_alumnos_nota
DROP INDEX idx_alumnos_nota;
  -- vuelve a HEAP_SCAN
```

La sintaxis es

```
CREATE [UNIQUE] INDEX nombre ON tabla (columna)
    [USING HASH | BPLUS_CLUSTERED | BPLUS_UNCLUSTERED]
DROP INDEX [IF EXISTS] nombre [ON tabla]
```

Y construye el índice recorriendo la tabla, por lo que funciona sobre datos ya cargados (escenario típico: cargar el CSV y después indexar). El índice sobrevive al reinicio del proceso porque queda en el manifiesto del catálogo y se reconstruye al abrirlo. El índice de la clave primaria no se puede eliminar, porque es el que hace cumplir la unicidad.

4.7. Transacciones y concurrencia

Transacciones en el motor. `BEGIN TRANSACTION`, `END TRANSACTION`, `COMMIT` y `ROLLBACK` los ejecuta el mismo `QueryExecutor` que atiende la interfaz. En la primera escritura sobre una tabla se guarda su estado previo y se toma el **bloqueo exclusivo** de esa tabla (*strict 2PL*: los bloqueos se liberan solo en `COMMIT` o `ROLLBACK`). `ROLLBACK` restaura los registros, reconstruye los índices y deshace el DDL de la transacción (`CREATE TABLE`, `CREATE INDEX` y `DROP TABLE`). Hay una única transacción activa por proceso, y el endpoint `GET /api/health` informa si hay una abierta (con su id, sentencias y tablas bloqueadas).

LockManager y demostración con hilos. `transactions/lock_manager.py` implementa bloqueos compartidos (PS) y exclusivos (PX) por recurso con el protocolo *strict 2PL*. Frente a un conflicto, la transacción espera hasta un *timeout* de 2 s y entonces lanza `RecursoBloqueado`; no hay detección de interbloqueos por grafo de espera. `demo_concurrencia.py` ejecuta transacciones concurrentes con hilos sobre un valor compartido `X = 100`: sin locks, dos transacciones (una que resta 10 y otra que suma 100) producen una *race condition* y se pierde una actualización (resultado distinto del esperado, 190); con el LockManager el resultado final es correcto.

Alcance. El proyecto no implementa recuperación ante fallos (registro de cambios *write-ahead* ni *UNDO/REDO* tras un *crash*); la atomicidad se logra restaurando el estado previo en memoria durante la sesión. El enunciado de la Parte 1 pide `BEGIN/END` y control de concurrencia, que sí están cubiertos.

4.8. Interfaz de usuario

La interfaz (React, Vite y TypeScript) cumple los cuatro paneles del enunciado más un gestor de tablas y un panel de mapa:

- **Panel de Archivos:** tablas registradas, tipo de almacenamiento, archivos con su tamaño, esquema, índices (incluido el R-Tree espacial) y número de registros. Incluye un *gestor de tablas* (pestañas "Importar CSV" y "Crear tabla") y, sobre la tabla seleccionada, un formulario para **crear índices** (columna + técnica + opción de unicidad) y un botón para **eliminar** cada índice existente.
- **Panel de Consultas:** editor SQL con ejemplos por caso de uso (incluidos los espaciales, `CREATE INDEX`, transacciones y `EXPLAIN`), ejecución con botón o `Ctrl+Enter` y mensajes de error del parser.
- **Panel de Resultados:** tabla de resultados; distingue entre una consulta sin filas y una mutación aplicada, y ofrece vista JSON cruda.
- **Panel de Plan de Ejecución:** plan lógico con la justificación de cada paso y traza de ejecución real con tiempos y filas por operador.
- **Panel de Mapa (Parte 2):** mapa interactivo con **Leaflet y teselas de OpenStreetMap** (zoom, pan y clic para fijar `mi_ubicacion`); modos rango, k-NN y polígono por clics; selector de métrica Haversine/Euclidiana; y superposición de resultados sobre el mapa. Ver Sección 5.11.

5. Parte 2: base de datos espacial

5.1. Alcance

La Parte 2 agrega un índice espacial para puntos 2D (latitud, longitud) con tres tipos de consulta: **rango** (por ejemplo, tiendas a menos de 5 km), k **vecinos más cercanos** y **intersección con polígonos**, con dos métricas de distancia: Euclidiana y Geodésica (Haversine).

5.2. Estructura del R-Tree

El R-Tree (`indexes/rtree.py`) es un árbol balanceado en el que cada entrada guarda un *rectángulo mínimo envolvente* (MBR):

- **Nodo hoja:** entradas `(MBR del objeto, RID)`.
- **Nodo interno:** entradas `(MBR, hijo)`, donde el MBR cubre todo el subárbol.
- **Parámetros:** $M = 16$ entradas máximas por nodo y $m = \lceil M/2 \rceil$ mínimas (excepto en la raíz).

A diferencia de un B+, los MBR de un mismo nivel **pueden solaparse**: el R-Tree no es una partición disjunta del espacio. El método `validate()` comprueba las invariantes: la raíz es hoja o tiene al menos dos hijos, todo nodo no raíz tiene entre m y M entradas, el MBR del padre contiene a los de sus hijos y todas las hojas están a la misma profundidad.

5.3. Inserción y división cuadrática

1. **ChooseLeaf:** se baja desde la raíz eligiendo el hijo cuyo MBR requiere la *menor ampliación de área* para incluir la entrada nueva (desempate: menor área).
2. Se inserta en la hoja. Si queda con $M + 1$ entradas se aplica **QuadraticSplit**:
   - *PickSeeds:* elige las dos entradas que más "desperdiciarían" si quedaran juntas, maximizando área(MBR($E_1 \cup E_2$)) − área(E_1) − área(E_2).
   - *PickNext:* reparte el resto asignando primero la entrada con mayor diferencia entre la ampliación en un grupo y en el otro, al grupo que menos crece (desempates por área y tamaño). Si a un grupo le faltan entradas para llegar a m, recibe todas las restantes.
3. **AdjustTree:** se propagan los MBR hacia arriba; si un nodo interno se divide, el proceso continúa, y si se divide la raíz, el árbol crece una unidad de altura.

Se eligió el split cuadrático (y no un R*-tree con reinserción forzada) por coherencia con el material de la asignatura, aun sabiendo que el R*-tree produce mejor llenado.

5.4. Carga masiva

Insertar punto por punto con `ChooseLeaf` produce mucho solapamiento y es lento para 10^5 puntos. `bulk_load` ordena los puntos por una clave que conserva la localidad espacial (celda de una rejilla de lado $\lfloor\sqrt{n}\rfloor$) y **empaqueta** las hojas al máximo, construyendo los niveles superiores hacia arriba. El reparto en cada nivel es *equilibrado* (cada nodo recibe b o $b + 1$ hijos) para que ningún nodo quede por debajo de m, de modo que el árbol resultante cumple las mismas invariantes. El catálogo usa la carga masiva al reconstruir los índices espaciales al abrir la base de datos.

5.5. Consulta por rango

La búsqueda por rango desciende solo por los nodos cuyo MBR se solapa con la ventana consultada y devuelve los candidatos. El índice *poda por MBR* y quien lo consume hace la comprobación exacta (distancia real menor que el radio, o pertenencia al polígono). Para una consulta circular de radio r (en metros) centrada en φ_0, la ventana es el MBR del círculo:

$$\Delta\varphi = \frac{r}{111\,320}, \qquad \Delta\lambda = \frac{r}{111\,320 \cdot \cos\varphi_0},$$

y cada candidato se verifica con la distancia real (Haversine o Euclidiana).

5.6. k vecinos más cercanos (best-first)

El k-NN usa una búsqueda *best-first* guiada por la distancia mínima de un punto Q a un rectángulo R:

$$\mathrm{MINDIST}(Q, R) = \sqrt{\sum_i d_i^2}, \qquad d_i = \begin{cases} Q_i - L_i & Q_i < L_i \\ Q_i - U_i & Q_i > U_i \\ 0 & \text{si } L_i \le Q_i \le U_i \end{cases}$$

1. Se coloca la raíz en una cola de prioridad ordenada por MINDIST.

2. Se extrae la entrada de menor MINDIST. Si ya hay k resultados y esa cota supera la k-ésima mejor distancia, se termina (**poda global**).
3. Si es una hoja, se calcula la distancia *real* a sus puntos y se actualiza el top-k; si es un nodo interno, se encolan sus hijos con su MINDIST.

La distancia real solo se calcula para los puntos de las hojas visitadas. La complejidad típica es $O(k \log n)$, con peor caso $O(n \log n)$.

5.7. Métricas de distancia y plano del índice

- **Euclidiana:** distancia en el plano $(x, y) = (\text{lon}, \text{lat})$, en grados.
- **Haversine** (en metros, con $R = 6\,371\,008.8\,\text{m}$):

$$a = \sin^2 \tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \tfrac{\Delta\lambda}{2}, \qquad d = 2R \arcsin\sqrt{a}.$$

Como el R-Tree es una estructura plana y Haversine vive en la esfera, el índice usa un plano propio por métrica: para Haversine, una *proyección equirectangular local* en metros, $x = (\lambda - \lambda_0) \cdot 111\,320 \cos\varphi_0$ e $y = (\varphi - \varphi_0) \cdot 111\,320$, con φ_0 la latitud central del conjunto de datos. Así MINDIST sigue siendo una cota inferior útil y el resultado es **exacto**: la cota solo decide el orden de visita de los nodos, y la distancia final se calcula siempre con Haversine sobre las coordenadas originales. El error de la proyección es despreciable para consultas locales; para datos que cubran continentes habría que indexar en 3D sobre la esfera.

5.8. Intersección con polígonos

La consulta usa el MBR del polígono como filtro en el R-Tree y confirma cada candidato con el algoritmo de *ray casting* (punto en polígono), con tolerancia para puntos sobre el borde.

5.9. Integración con el motor

`SpatialIndex` (`spatial/index.py`) envuelve el R-Tree y resuelve las coordenadas de cada RID; ofrece inserción, carga masiva, borrado, reconstrucción, `range_search`, `knn`, `polygon_search`, `stats` y `validate`. El `SpatialExecutor` (`query/spatial_queries.py`) detecta en la consulta un predicado de distancia, un `ORDER BY` por distancia con `LIMIT` o un predicado de polígono, y elige la ruta `RTREE_RANGE_SCAN`, `RTREE_KNN` o `RTREE_POLYGON`. Si no existe índice espacial, la consulta se resuelve con `HEAP_SCAN` más `FILTER`, calculando la distancia fila por fila. Los resultados incluyen las columnas calculadas `_lat`, `_lon` y `_distance`, que usa el panel de mapa para resaltar los puntos.

5.10. Extensión del lenguaje SQL

El lenguaje SQL se extendió para soportar consultas espaciales mediante las funciones `distancia`, `POINT` y `dentro_de`. El orden de los argumentos de `POINT` es latitud, longitud, y el radio se expresa en metros para Haversine y en grados para la métrica Euclidiana. La sintaxis implementada es:

```
CREATE INDEX idx_tiendas_ubicacion ON tiendas (lat, lon) USING RTREE;

-- rango
SELECT * FROM tiendas
WHERE distancia(lat, POINT(-12.0464, -77.0428)) < 5000;
```

```
-- k-NN
SELECT nombre FROM restaurantes
ORDER BY distancia(lat, POINT(-12.0464, -77.0428)) LIMIT 10;

-- intersección con polígono
SELECT * FROM tiendas
WHERE dentro_de(lat, POLYGON((-12.20 -77.20, -12.20 -76.90,
                          -12.00 -76.90, -12.00 -77.20)));

-- métrica Euclidiana
SELECT * FROM tiendas
WHERE distancia(lat, POINT(-12.04, -77.04), EUCLIDEAN) < 0.05;
```

5.11. Visualización en mapa

El panel de mapa usa **Leaflet** con teselas de **OpenStreetMap**: mapa base real con zoom y desplazamiento (pan), y clic sobre el mapa para fijar el punto de consulta (`mi_ubicacion`). Ofrece tres modos de búsqueda y un selector de métrica (Haversine o Euclidiana). Sobre el mapa se dibuja:

- **Rango:** el círculo del radio solicitado (en metros reales) y se resaltan en naranja los puntos que caen dentro; los candidatos descartados se atenúan.
- **k-NN:** se traza una línea desde el punto de consulta a cada vecino y se numera cada resultado del 1 al k sobre el mapa.
- **Polígono:** cada clic añade un vértice; con 3 o más se dibuja el contorno del anillo y se resaltan los puntos interiores. Hay un botón para borrar los vértices.

El panel también muestra la ruta de acceso utilizada (`RTREE_RANGE_SCAN`, `RTREE_KNN` o `RTREE_POLYGON`), el nombre del índice espacial, la cantidad de resultados, el tiempo de la consulta y las columnas calculadas `_lat`, `_lon` y `_distance`. Adicionalmente hay una tabla de resultados clicable que centra el mapa en cada punto. Sin conexión a OpenStreetMap, el mapa base no carga y se avisa al usuario, pero las búsquedas y los puntos siguen funcionando sobre el lienzo vacío.

6. Evaluación experimental de la Parte 1

6.1. Metodología

- **Datos y semilla:** registros de 43 bytes, semilla 42, el mismo dataset y orden de inserción para todas las técnicas.
- **Archivos:** Heap File vs Archivo Secuencial con 1 000, 10 000 y 100 000 registros: inserción, búsqueda por clave primaria (aciertos y fallos), borrado, reorganización, espacio en disco y desperdicio.
- **Índices:** B+ agrupado, B+ no agrupado y Hash Extendible con 1 000, 10 000 y 50 000 claves: construcción, igualdad exacta, rango, recorrido ordenado, inserciones y borrados, y espacio. Orden del B+ y capacidad de bucket del hash: 64; 500 consultas de igualdad, 100 de rango y 200 mutaciones; promedio de 3 repeticiones. Se incluye la **línea base sin índice** (búsqueda lineal).
- **Carga a 100 000 registros:** la inserción uno a uno del Archivo Secuencial crece con el tamaño, por lo que a esa escala se usa carga masiva, que escribe el archivo con el mismo

formato binario que la API pública y se verifica *byte a byte* en las pruebas. Las inserciones y borrados uno a uno se miden hasta 10 000 registros (archivos) y 50 000 (índices).

- **Entorno:** Windows 11 y Python 3.13.7, con una capa que intercepta la E/S de archivos. Cada corrida mide su calibración (lectura y escritura de un registro de 43 bytes: cerca de 2.56 ms y 0.46 ms), por lo que los **tiempos absolutos están inflados**. Las conclusiones se apoyan en las diferencias relativas, la tendencia con N y el análisis de complejidad.

6.2. Heap File vs Archivo Secuencial Paginado

Métrica	N	Heap File	Archivo Secuencial
Inserción (μs/registro)	1 000	15 393	15 232
	10 000	14 826	16 375
Borrado (μs/operación)	1 000	1 950	15 640
	10 000	2 094	12 630
Búsqueda por PK acertada (μs)	1 000	3 438	322.0
	10 000	36 597	246.0
	100 000	317 507	450.8
Búsqueda por PK fallida (μs)	1 000	4 076	577.3
	10 000	38 597	364.6
	100 000	326 892	628.8
Reorganización (ms)	1 000	—	5.9
	10 000	—	36.5
Carga masiva (ms totales)	1 000	36.5	6.1
	10 000	25.4	26.9
	100 000	258.3	316.4

Tabla 4: Heap File vs Archivo Secuencial Paginado: tiempos (semilla 42, registros de 43 bytes).

N	Disco Heap (B)	Disco Sec. (B)	Desperdicio Heap (B)	Desperdicio Sec. (B)
1 000	44 935	43 000	1 935	1 548
10 000	433 010	430 000	3 010	16 082
100 000	4 301 505	4 300 000	1 505	n/m

Tabla 5: Espacio en disco y desperdicio (n/m: no medido a esa escala). En el Archivo Secuencial el desperdicio son los *tombstones* acumulados justo antes de reorganizar.

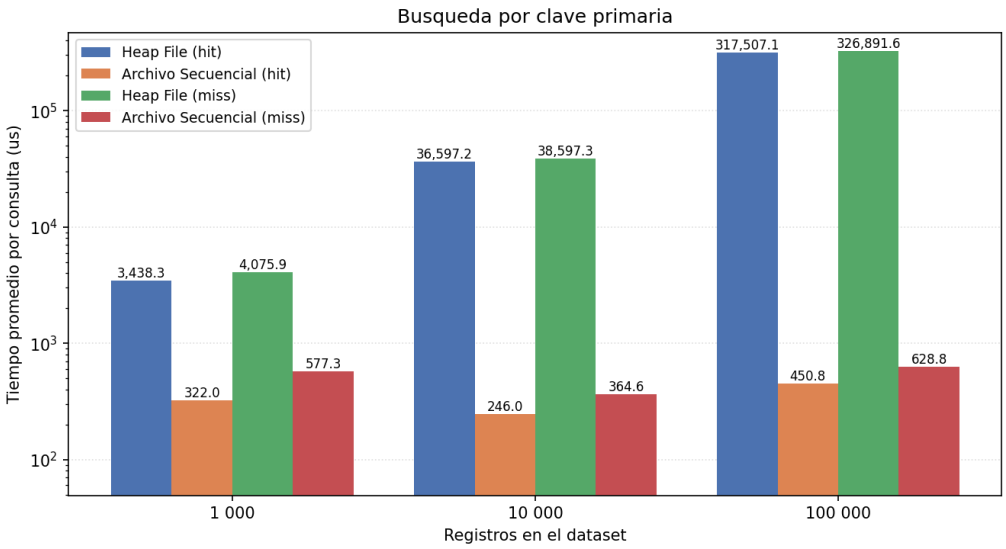


Figura 2: Búsqueda por clave primaria: el Heap File crece linealmente y el Archivo Secuencial se mantiene casi plano.

Interpretación.

1. **Inserción: empate técnico.** Ambas estructuras insertan en tiempo prácticamente constante (cerca de 15 ms por registro en este entorno, dominado por el costo de E/S). El Archivo Secuencial no degrada porque reorganiza periódicamente y paga el costo de forma amortizada.
2. **Búsqueda: el Archivo Secuencial es órdenes de magnitud mejor.** La búsqueda binaria se mantiene casi plana al crecer el dataset, mientras que el *full scan* del Heap File crece linealmente. Con 100 000 registros la diferencia es de unas 704 veces, coherente con $O(\log N)$ frente a $O(N)$.
3. **Borrado: el Heap File es entre 6–8 veces más barato**, porque solo marca un *tombstone* y actualiza la *free-list*; el Archivo Secuencial puede disparar además una reorganización completa.
4. **Reutilización de espacio.** Tras borrar el 30 % de los registros, 300 de 300 (con $N = 1\,000$) y 3 000 de 3 000 (con $N = 10\,000$) inserciones posteriores reutilizaron un slot liberado, sin costo adicional.
5. **Reorganización:** costo lineal y predecible; multiplicar N por 10 multiplica el tiempo por 6.1.
6. **Espacio:** prácticamente empatados. El Heap File desperdicia solo el resto de la última página; el Archivo Secuencial usa exactamente $N \times 43$ bytes tras reorganizar, pero acumula tombstones entre reorganizaciones.

	Heap File	Archivo Secuencial
Ventajas	Inserción y borrado simples y baratos; reutilización de espacio sin compactar; ideal para *scans* completos y cargas masivas	Búsqueda por clave en $O(\log N)$; mantiene el orden físico; sin desperdicio tras reorganizar
Desventajas	Búsqueda por clave $O(N)$; espacio en múltiplos de página	Borrado costoso; reorganizaciones frecuentes en la carga; invalida los RID ya indexados
Cuándo usarla	Muchas altas y bajas, consultas poco selectivas, o cuando se indexa por fuera (B+ no agrupado o hash)	Consultas por clave primaria o por rango en tablas que cambian poco entre reorganizaciones

Tabla 6: Resumen comparativo de las técnicas de almacenamiento.

6.3. B+ agrupado vs B+ no agrupado vs Hash Extendible

Métrica	N	B+ agrupado	B+ no agrupado	Hash extendible
Construcción (ms)	1 000	36.5	18.6	8.7
	10 000	568.8	379.6	113.0
	50 000	4 003	2 580	943.5
Igualdad exacta, solo el índice (μs)	1 000	10.40	1.04	1.05
	10 000	10.76	1.68	1.26
	50 000	10.96	1.97	1.43
Igualdad exacta + traer la fila (μs)	1 000	9.20	149.58	134.51
	10 000	9.87	183.95	146.46
	50 000	11.63	135.11	127.34
Igualdad exacta fallida (μs)	1 000	2.27	0.74	0.85
	10 000	3.76	1.12	0.92
	50 000	2.45	1.45	1.08
Rango, solo el índice (μs)	1 000	89.0	5.8	n/a
	10 000	705.8	65.2	n/a
	50 000	3 682	380.8	n/a
Rango + traer cada fila del Heap (μs)	1 000	83.1	1 436	n/a
	10 000	676.8	12 145	n/a
	50 000	4 104	61 341	n/a
Recorrido ordenado (μs)	1 000	6 722	310	n/a
	10 000	74 956	9 258	n/a
	50 000	449 827	60 714	n/a
Inserción (μs)	1 000	14.04	6.30	4.61
	10 000	20.01	12.58	1.25
	50 000	22.50	15.77	2.23
Borrado (μs)	1 000	9.19	8.71	2.85
	10 000	18.37	14.15	2.93
	50 000	18.50	28.05	4.11

Tabla 7: Estructuras de indexación (semilla 42, orden B+ = 64, bucket = 64, promedio de 3 repeticiones). n/a: el hash no soporta rangos ni recorrido ordenado.

N	Búsqueda lineal (μs)	B+ agrupado	B+ no agrupado	Hash
1 000	74.3	7×	71×	71×
10 000	3 583	333×	2 137×	2 835×
50 000	25 509	2 327×	12 956×	17 788×

Tabla 8: Mejora respecto a la línea base sin índice (igualdad exacta, solo el índice).

N	B+ agrupado (B)	B+ no agrupado (B)	Hash (B)	Registros dentro del agrupado (B)
1 000	44 773	25 056	24 316	39 000
10 000	455 775	250 224	245 143	390 000
50 000	2 285 197	1 265 838	1 234 544	1 950 000

Tabla 9: Tamaño del índice (pickle del índice: aproximación portable) y bytes que son registros completos en las hojas del B+ agrupado.

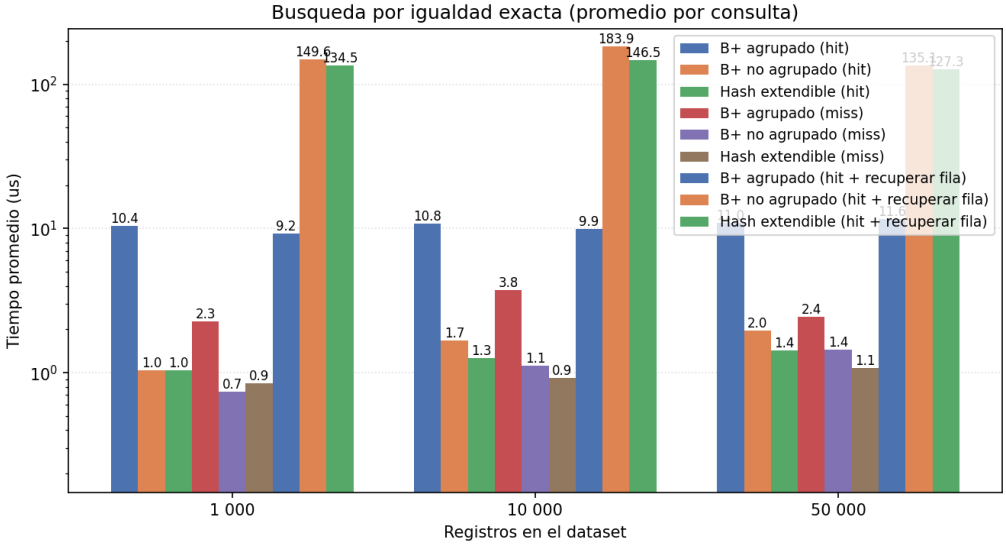


Figura 3: Igualdad exacta por índice.

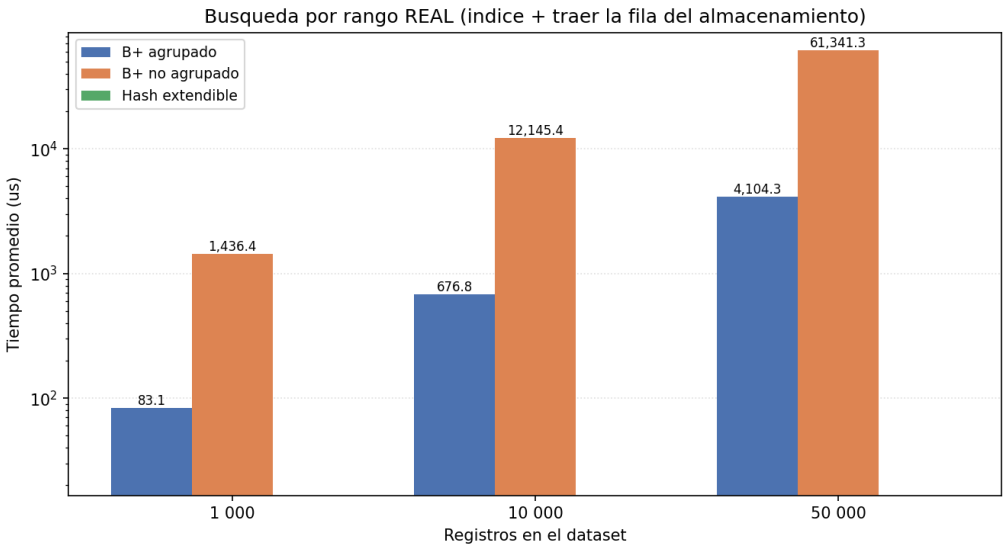


Figura 4: Búsqueda por rango: comparación real (índice más traer la fila).

Interpretación.

1. **Construcción:** el hash es el más barato (944 ms con 50 000 claves frente a 2 580 ms del B+ no agrupado y 4 003 ms del agrupado, es decir, 2.7 y 4.2 veces más). El B+ agrupado paga además la copia del registro completo en cada hoja.
2. **Igualdad exacta: hash > B+ no agrupado > B+ agrupado** (1.43, 1.97 y 10.96 μs con 50 000 claves, solo el índice). Frente a la búsqueda lineal, la mejora es de 17 788, 12 956 y 2 327 veces, y crece con N. Si se cuenta *traer la fila*, el orden se invierte: el agrupado es unas 12 veces más rápido, porque ya tiene el registro en la hoja y los otros dos pagan una lectura por RID en el Heap File.
3. **Rango:** el hash no puede resolverlo. Midiendo solo el recorrido de hojas, ambos B+ empatan; cuando la consulta necesita las filas (el caso real), el agrupado gana entre 15–18 veces (15 veces con 50 000 claves: 4 104 μs frente a 61 341 μs), pues se resuelve como una lectura secuencial de hojas en vez de una lectura por cada RID.
4. **Mutaciones:** el hash es el más estable (2.23 μs por inserción y 4.11 μs por borrado con 50 000 claves; el B+ agrupado inserta unas 10 veces más lento). Los B+ pagan el rebalanceo.
5. **Espacio:** el B+ agrupado es el más grande (1.9 veces el del hash con 50 000 claves), porque 85.3 % de su tamaño son los registros completos que guarda en las hojas; el B+ no agrupado y el hash solo guardan clave y RID y quedan muy cerca entre sí.
6. **Recorrido ordenado:** en esta medición el B+ agrupado resulta unas 7.4 veces más lento (449 827 frente a 60 714 μs con 50 000 claves) porque su capa de acceso entrega copias defensivas de cada registro; no es una limitación del algoritmo.

Aclaración metodológica sobre el rango del B+ agrupado. Una primera versión del benchmark mostraba al agrupado *perdiendo* en rango, lo contrario de lo que predice la teoría. Los índices estaban correctos: se le pedía al no agrupado solo el recorrido (RIDs) y al agrupado el recorrido más la copia de cada registro completo. La Tabla 10 separa las variantes: sin la copia defensiva ambos recorridos empatan; con la recuperación real desde el Heap File gana el agrupado; y la variante contra un `dict` en memoria se conserva a propósito como contraste, porque en un motor puramente en memoria mover un RID es más barato que copiar un registro (3.6–4.5 veces a favor del no agrupado).

Variante medida	B+ agrupado	B+ no agrupado
Solo el índice (sin traer la fila)	3 682	380.8
Recorrido de hojas sin copia defensiva	374.5	379.9
Índice + traer la fila desde el Heap File (real)	4 104	61 341
Índice + traer la fila desde un `dict` (contraste)	3 622	1 004

Tabla 10: Rango del 1 % del dataset, $N = 50\,000$ (μs por consulta). Qué se mide decide quién gana.

	B+ agrupado	B+ no agrupado	Hash extendible
Contenido de las hojas	Registro completo	`(clave, RID)`	`(clave, RID)` en buckets
Rango y `ORDER BY`	Sí; sin lecturas extra al Heap	Sí; una lectura por RID	No
Espacio y construcción	Mayores	Intermedios	Menores
Cuándo usarlo	Tabla ordenada por esa clave, consultas de rango o `SELECT *`	Índice secundario con igualdad y rango	Columna usada casi solo en igualdad exacta

Tabla 11: Resumen comparativo de las estructuras de indexación.

Conclusiones: cuándo usar cada técnica.

- Igualdad exacta con muchas lecturas: **Hash Extendible**.
- Igualdad y rangos sobre la misma columna: **B+** (agrupado si la tabla está ordenada por esa clave y se necesita traer las filas; no agrupado como índice secundario).
- Tabla con muchas altas y bajas y consultas por clave: **Heap File** con índice B+ o hash.
- Tabla consultada casi siempre por clave primaria y con pocas mutaciones: **Archivo Secuencial Paginado**.
- `ORDER BY` sin índice: **External Sort**. `GROUP BY` y equi-`JOIN` sobre datos grandes: **External Hashing**.

6.4. Resumen de resultados de la Parte 1

Consulta	Mejor técnica	Mejora observada	Por qué
Búsqueda por PK	Archivo Secuencial	$\sim 700\times$ sobre Heap File	$O(\log N)$ vs $O(N)$
Igualdad exacta	Hash Extendible	$17\,788\times$ sobre scan lineal	$O(1)$ promedio
Rango	B+ agrupado	$15\times$ sobre B+ no agrupado	Hoja = registro, sin fetch
ORDERBY con índice	B+	evita External Sort	Hojas enlazadas ordenadas
ORDERBY sin índice	External Sort	—	memoria acotada
GROUPBY / JOIN	External Hashing	—	$O(N + M)$ promedio

Tabla 12: Resumen de resultados de la Parte 1.

7. Evaluación experimental de la Parte 2

7.1. Metodología

La evaluación compara tres alternativas para consultas espaciales: **búsqueda secuencial**, **R-Tree propio** y **GiST de PostgreSQL con PostGIS**. Se utilizaron datasets sintéticos de 1 000, 10 000 y 100 000 puntos, generados con semilla fija 42 y con focos de densidad para producir solapamiento entre los MBR del R-Tree.

Para cada configuración se ejecutaron 100 consultas (más 3 de calentamiento no cronometradas) y se reportó el tiempo promedio en milisegundos. Se evaluaron consultas por rango con radios de 1, 5 y 10 km, así como consultas de k vecinos más cercanos con $k \in \{10, 50, 100\}$. También se evaluó el tiempo de construcción y el espacio utilizado por las estructuras.

El baseline de PostgreSQL utiliza **PostGIS 3.4.2 sobre PostgreSQL 16.15 (Ubuntu 24.04)**, con geometrías de tipo `Point` en SRID 4326 y un índice GiST 2D. La corrida se ejecutó en Linux con Python 3.12.3. El espacio del R-Tree se mide sobre su serialización binaria completa (`RTree.serialize()`) y el del GiST con `pg_relation_size(índice)`: métricas comparables entre sí. El tamaño de la tabla se reporta aparte como contexto.

7.2. Consultas por rango

Para evaluar las consultas por rango se utilizaron radios de 1 km, 5 km y 10 km. La búsqueda secuencial calcula la distancia Haversine a todos los puntos del dataset, mientras que el R-Tree poda los rectángulos mínimos (MBR) que no intersecan el círculo de búsqueda. PostgreSQL utiliza el operador `ST_DWithin` del índice GiST.

N	Secuencial (ms)	R-Tree (ms)	GiST/PostGIS (ms)
1 000	0.817	**0.038**	0.163
10 000	8.289	0.242	**0.197**
100 000	88.545	2.055	**0.342**

Tabla 13: Consulta por rango con radio de 1 km.

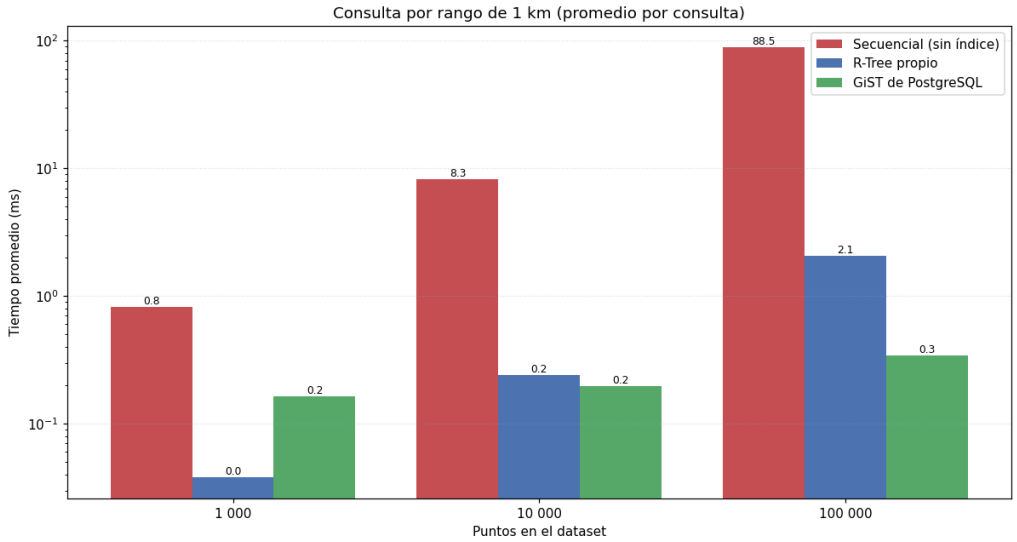


Figura 5: Consulta por rango con radio de 1 km en los tres tamaños de dataset.

N	Secuencial (ms)	R-Tree (ms)	GiST/PostGIS (ms)
1 000	0.807	**0.077**	0.202
10 000	8.337	0.778	**0.588**
100 000	88.845	11.630	**10.032**

Tabla 14: Consulta por rango con radio de 5 km.

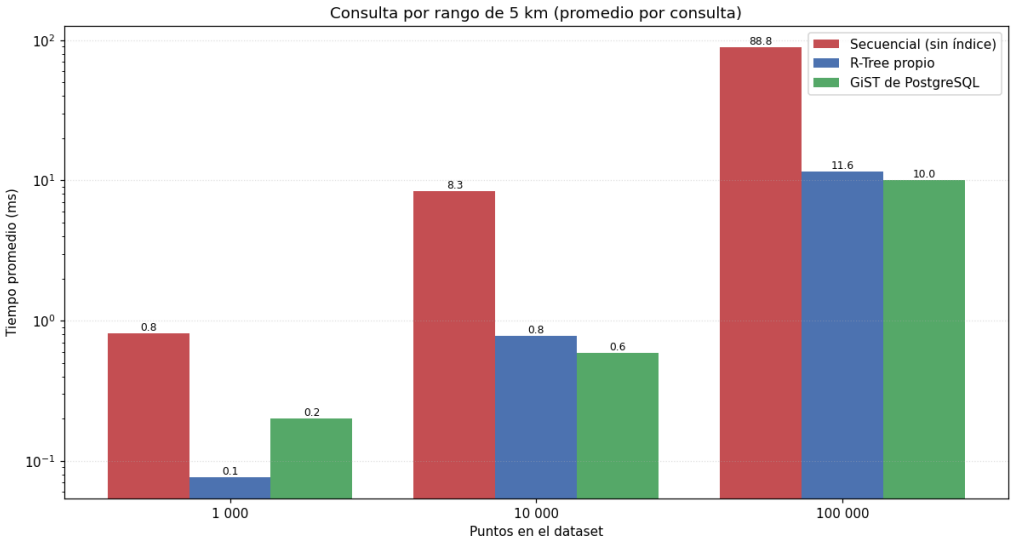


Figura 6: Consulta por rango con radio de 5 km en los tres tamaños de dataset.

N	Secuencial (ms)	R-Tree (ms)	GiST/PostGIS (ms)
1 000	0.838	**0.188**	0.290
10 000	8.437	2.462	**1.395**
100 000	89.167	38.093	43.485

Tabla 15: Consulta por rango con radio de 10 km.

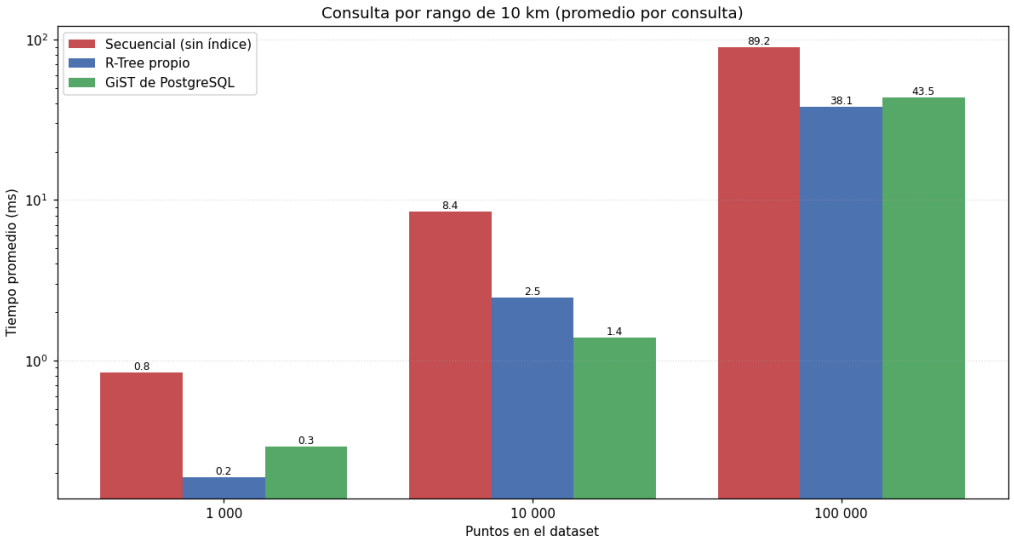


Figura 7: Consulta por rango con radio de 10 km en los tres tamaños de dataset.

7.3. k vecinos más cercanos

Para evaluar el operador de vecinos más cercanos se utilizaron $k = 10$, $k = 50$ y $k = 100$. La búsqueda secuencial calcula la distancia a todos los puntos para determinar los vecinos, mientras que el R-Tree utiliza una estrategia *best-first* guiada por MINDIST. PostgreSQL utiliza el operador espacial del índice GiST.

N	Secuencial (ms)	R-Tree (ms)	GiST/PostGIS (ms)
1 000	0.904	**0.418**	0.220
10 000	9.494	1.921	**0.237**
100 000	116.289	11.350	**0.258**

Tabla 16: k-NN con $k = 10$ por tamaño de dataset.

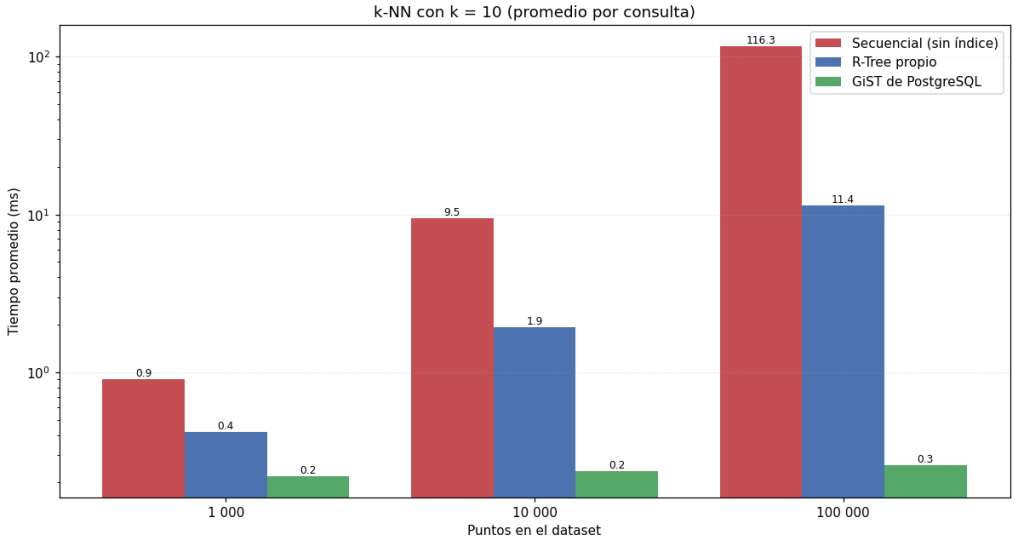


Figura 8: k-NN con $k = 10$ en los tres tamaños de dataset.

N	Secuencial (ms)	R-Tree (ms)	GiST/PostGIS (ms)
1 000	0.899	**0.550**	0.352
10 000	10.387	2.269	**0.396**
100 000	148.088	11.033	**0.433**

Tabla 17: k-NN con $k = 50$ por tamaño de dataset.

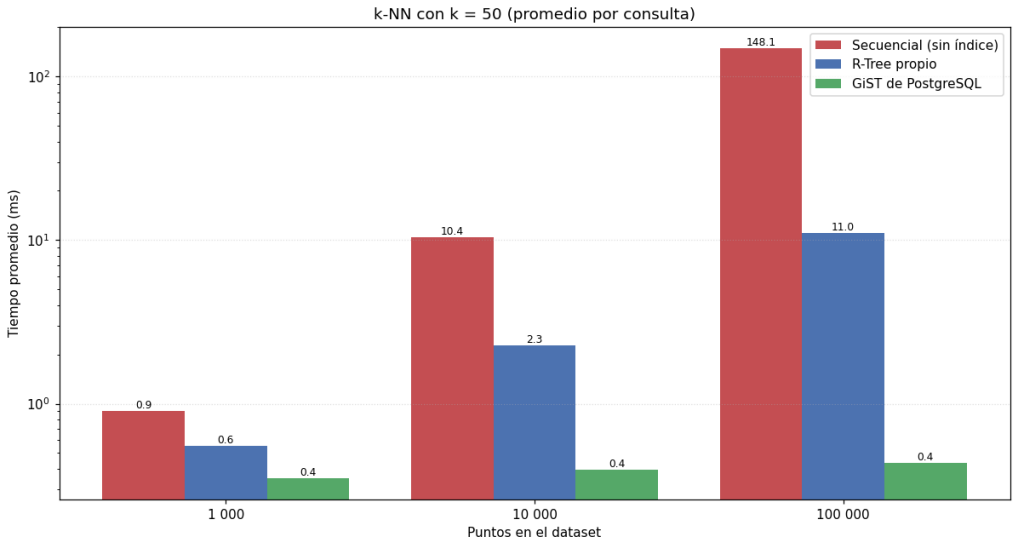


Figura 9: k-NN con $k = 50$ en los tres tamaños de dataset.

N	Secuencial (ms)	R-Tree (ms)	GiST/PostGIS (ms)
1 000	0.903	**0.658**	0.512
10 000	9.914	2.706	**0.619**
100 000	174.897	12.949	**0.633**

Tabla 18: k-NN con $k = 100$ por tamaño de dataset.

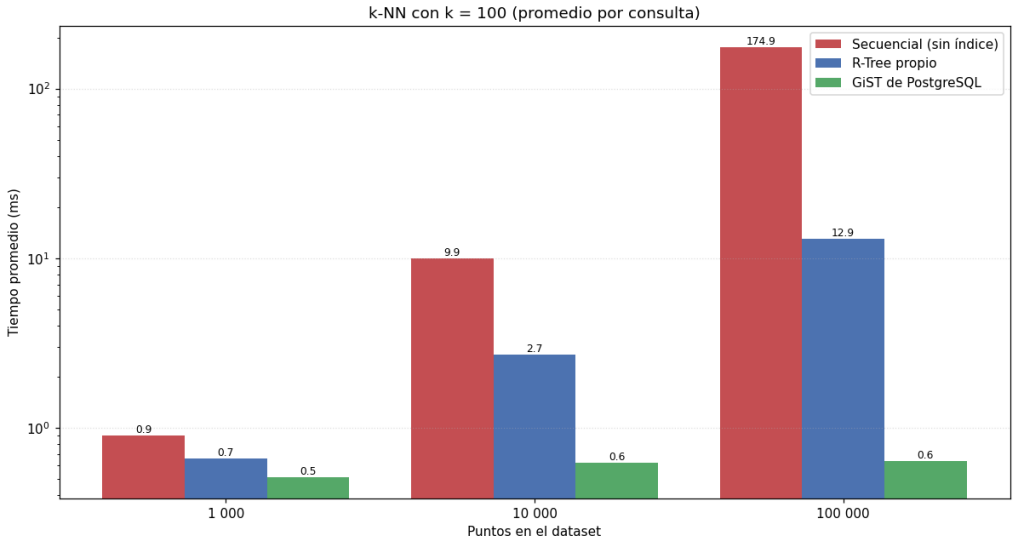


Figura 10: k-NN con $k = 100$ en los tres tamaños de dataset.

7.4. Construcción del índice y espacio

Además del tiempo de consulta, el enunciado pide medir el tiempo de construcción del índice y su uso de espacio. La Tabla 19 compara el tiempo de construcción y el espacio ocupado por cada estructura en los tres tamaños de dataset. El espacio del R-Tree es el tamaño de su serialización binaria completa (todos los nodos y todas las entradas: MBR de 32 B más puntero/RID de 8 B); el del GiST es `pg_relation_size` del índice, que es la métrica comparable.

N	R-Tree: construc. (ms)	GiST: construc. (ms)	R-Tree: espacio	GiST: espacio
1 000	4.5	9.5	42.0 KiB	48.0 KiB
10 000	52.2	49.0	419.4 KiB	384.0 KiB
100 000	715.5	287.0	4.09 MiB	3.94 MiB

Tabla 19: Construcción y espacio del índice espacial (R-Tree serializado frente a `pg_relation_size` del GiST, ambos índices medidos solos).

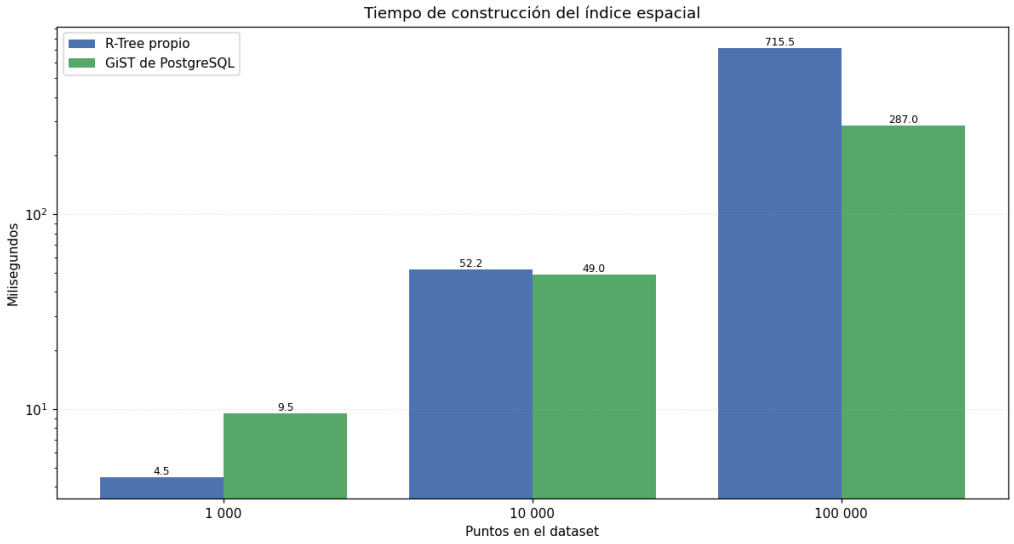


Figura 11: Tiempo de construcción del índice espacial.

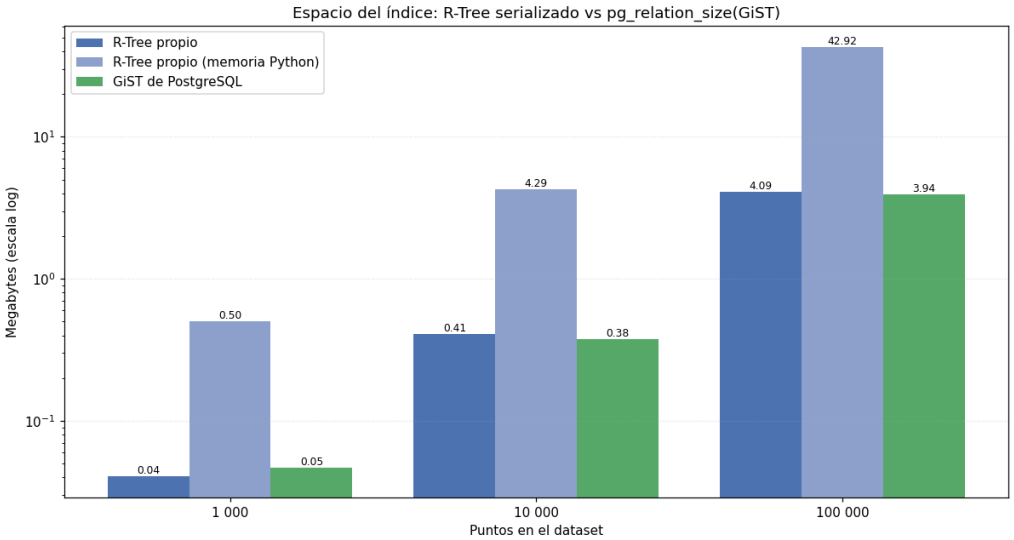


Figura 12: Espacio ocupado por el índice espacial.

7.5. Intersección con polígonos

La consulta de intersección con polígonos se evaluó solo con el R-Tree, porque es el único que la soporta (la búsqueda secuencial tendría que recorrer todos los puntos y comprobar punto-en-polígono con ray casting). Se usó un anillo de 4 vértices centrado en el dataset.

N	Tiempo (ms)	Resultados
1 000	0.099	360
10 000	0.717	2 780
100 000	11.127	29 480

Tabla 20: Intersección con un polígono de 4 vértices.

7.6. Interpretación de resultados

Los resultados muestran que ambas estructuras indexadas evitan el crecimiento lineal de la búsqueda secuencial. Para 100 000 puntos, una consulta de rango de 1 km tarda 88.545 ms con el recorrido secuencial, frente a 2.055 ms con el R-Tree propio y 0.342 ms con GiST/PostGIS. Por tanto, el R-Tree propio obtiene una mejora de aproximadamente 43.1× respecto al recorrido secuencial, mientras que GiST alcanza 258.7×.

La ventaja del índice propio disminuye a medida que aumenta el radio de la consulta. Con 1 km, el R-Tree reduce considerablemente la cantidad de candidatos mediante los MBR. Para 5 km y 10 km, la región consultada cubre una mayor cantidad de puntos, por lo que la poda espacial es menos efectiva. Aun así, para 100 000 puntos el R-Tree mantiene mejoras de 7.6× y 2.3×, respectivamente (GiST obtiene 8.9× y 2.1×). En radios grandes, el costo de devolver los miles de resultados domina sobre el costo de la búsqueda, así que los dos índices quedan casi parejos.

En las consultas k-NN, el R-Tree propio también supera ampliamente al recorrido secuencial. Para $k = 10$, el tiempo disminuye de 116.289 ms a 11.350 ms, equivalente a una mejora de 10.2×. Sin embargo, GiST/PostGIS obtiene tiempos considerablemente menores, especialmente para datasets grandes (0.258 ms con $N = 100\,000$, una mejora de 451.2× frente al secuencial). Con $k = 50$ y $k = 100$ el patrón se repite: el R-Tree propio mejora 13.4× y 13.5× frente al secuencial, y GiST alcanza 341.8× y 276.5×.

Con datasets pequeños ($N = 1\,000$) el R-Tree propio *gana* a PostGIS en consultas por rango (4.3× más rápido con 1 km, 2.6× con 5 km y 1.5× con 10 km), porque el costo de planificación y de ida y vuelta al servidor pesa más que la propia búsqueda. En cambio, en k-NN con $N = 1\,000$ GiST ya es más rápido (1.9×, 1.6× y 1.3× para $k = 10$, 50 y 100), porque su operador `<->` está integrado en el optimizador y escrito en C.

La diferencia frente a PostgreSQL es esperable porque GiST/PostGIS es una implementación especializada y optimizada para consultas espaciales, escrita en C. Por ello, el resultado del R-Tree propio demuestra principalmente que la estructura implementada consigue la poda espacial esperada y ofrece una mejora significativa frente al recorrido secuencial, aunque todavía existe margen de optimización, especialmente en el cálculo de Haversine (que hoy se hace en Python puro).

El espacio de los dos índices es equivalente: ambos ocupan alrededor de 41–43 B por punto. Con 100 000 puntos, el R-Tree serializado ocupa 4.09 MiB (42.9 B por punto) y el GiST 3.94 MiB (41.3 B por punto): es el costo de un MBR más un puntero por entrada. La construcción, en cambio, favorece al R-Tree en datasets pequeños (4.5 ms frente a 9.5 ms con $N = 1\,000$) y al GiST en datasets grandes (287 ms frente a 715.5 ms con $N = 100\,000$), coherente con que la carga masiva del R-Tree se hace en Python.

7.7. Comparación cualitativa

	R-Tree propio	GiST de PostgreSQL
Ventajas	Sin dependencias externas; integrado con el motor; gana en rango con datasets pequeños (4.3× más rápido que PostGIS con $N = 1\,000$)	Implementado en C; índice 2D sobre `geometry(Point,4326)`; integrado en el optimizador; 258.7× más rápido en rango y 451.2× en k-NN con $N = 100\,000$
Desventajas	Cálculo de Haversine en Python; MBR pueden solaparse con datos sesgados; índice en memoria	Requiere instalar PostgreSQL + PostGIS; coste de planificación fijo
Cuándo usarla	Datasets pequeños, prototipos sin servidor, motores embebidos	Producción, consultas espaciales variadas, volúmenes grandes

Tabla 21: Comparación cualitativa de las técnicas espaciales.

8. Pruebas y verificación

El proyecto cuenta con una suite de pruebas automatizadas con `pytest`:

- **Unitarias** por estructura: registros, Heap File, Archivo Secuencial, B+ (base, agrupado y no agrupado), Hash Extendible, External Sort, External Hashing y R-Tree.
- **Integración y *end-to-end*:** parser, planner, ejecutor SQL, almacenamiento con índices, transacciones, tablas dinámicas (`test_dynamic_tables.py`), dinámica de la Parte 1 (`test_dinamica_parte1.py`), importación de CSV y el script de consultas proporcionado por el profesor (con comentarios y punto y coma), ejecutado literalmente en `tests/test_profesor_script.py`.
- **Espaciales:** MINDIST (incluida la propiedad de cota inferior), operaciones de MBR, invariantes del árbol, y rango, k-NN y polígono verificados contra **fuerza bruta**: el índice devuelve exactamente el mismo conjunto que recorrer todos los puntos. Incluye `test_spatial_sql.py`, `test_rtree.py` y `test_cumplimiento.py`.
- **Metodología de benchmarks:** la carga masiva produce un archivo idéntico byte a byte al que genera la API pública, y se valida que los benchmarks incluyan la línea base sin índice.

La suite completa recoge **425 pruebas** en **23 archivos** (que cubren `storage/`, `indexes/`, `operators/`, `query/`, `transactions/` y `spatial/`) y termina sin fallos. La verificación se ejecuta con:

```
python -m pytest -q
# 425 tests collected in 3.53s
```

Adicionalmente, el script SQL proporcionado por la cátedra se ejecuta literalmente en `tests/test_profesor_script.py` como prueba *end-to-end*.

9. Limitaciones y trabajo futuro

1. **Sin recuperación ante fallos:** no hay registro *write-ahead* ni *UNDO/REDO*; hay una sola transacción activa por proceso y los conflictos de bloqueo se resuelven por *timeout*, sin detección de interbloqueos.
2. **Esquema declarado a mano:** el motor no infiere el esquema desde un CSV.
3. **Entorno de medición:** los tiempos absolutos están inflados por la capa de E/S; las comparaciones son relativas. La comparación de mutaciones se limita a 50 000 registros por el costo de E/S.
4. **Copia defensiva en el B+ agrupado:** devolver referencias en lugar de copias reduciría su desventaja en igualdad, rango y recorrido ordenado.
5. **Reorganización frecuente del Archivo Secuencial** con `.main` pequeño; un umbral proporcional reduciría ese costo.
6. **Índices en memoria:** los índices (incluido el R-Tree) se reconstruyen al abrir el catálogo; un índice paginado en disco sería el paso siguiente.
7. **R-Tree:** el split cuadrático puede generar MBR muy solapados en datos sesgados (un R*-tree lo mitigaría), y el k-NN calcula Haversine en Python, que es la parte más lenta del índice.

10. Conclusiones

Se construyó un MiniGestor de Base de Datos Multimodal que integra almacenamiento físico, estructuras de indexación, algoritmos externos, procesamiento SQL, transacciones, control de concurrencia, una interfaz web y capacidades de consulta espacial.

En la Parte 1, los experimentos muestran diferencias claras entre las estructuras de almacenamiento e indexación. El Archivo Secuencial Paginado mantiene un tiempo de búsqueda por clave prácticamente estable frente al crecimiento del dataset ($\sim 700\times$ más rápido que el Heap File con 100 000 registros), mientras que el Heap File requiere un recorrido lineal. En indexación, el Hash Extendible resulta el más conveniente para búsquedas de igualdad (17 788× sobre la búsqueda lineal a 50 000 claves), mientras que los árboles B+ permiten además resolver consultas por rango y recorridos ordenados.

La evaluación también mostró que el costo de una estructura de indexación depende de qué parte de la consulta se mida. En particular, el B+ agrupado puede almacenar los registros directamente en sus hojas, reduciendo el costo de recuperar las filas en consultas por rango (15× más rápido que el no agrupado cuando se cuentan las lecturas al Heap File), a cambio de un mayor consumo de espacio. Por esta razón, los benchmarks separan el costo del índice del costo de acceder a los datos.

En la Parte 2, el R-Tree propio reduce significativamente el costo de las consultas espaciales respecto a un recorrido secuencial. Con 100 000 puntos, una consulta de rango de 1 km pasa de 88.545 ms a 2.055 ms (43× de mejora), mientras que una consulta k-NN con $k = 10$ pasa de 116.289 ms a 11.350 ms (10×). Esto confirma experimentalmente el beneficio de la poda mediante MBR y de la búsqueda *best-first*. Además, con datasets pequeños el R-Tree propio llega a superar a PostGIS en consultas por rango (4.3× más rápido con $N = 1\,000$ y radio de 1 km), lo que muestra que un índice integrado en el motor y sin ida y vuelta al servidor es competitivo cuando el dataset cabe en memoria.

Finalmente, GiST/PostGIS presenta mejores tiempos que el R-Tree propio en los experimentos espaciales, especialmente en k-NN (hasta 451× sobre el recorrido secuencial con 100 000 puntos).

Esto evidencia que la implementación realizada cumple correctamente el objetivo académico de construir y evaluar un índice espacial, pero también muestra oportunidades de optimización para una implementación futura, especialmente en el cálculo de distancias y en la reducción del solapamiento de los MBR.

En conjunto, los experimentos permiten relacionar directamente las decisiones de diseño con su comportamiento observado y muestran cómo la elección de almacenamiento e índice debe depender del tipo de consulta que predomina en una aplicación.

A. Cómo reproducir los experimentos

```
# Parte 1
python benchmarks/benchmark_heap_vs_sequential.py --sizes 1000 10000 100000
python -m benchmarks.benchmark_indexes --sizes 1000 10000 50000
python -m benchmarks.generate_charts

# Parte 2
python -m benchmarks.benchmark_spatial --sizes 1000 10000 100000 --pg-mode postgis
python -m benchmarks.generate_spatial_charts

# Tablas del informe (desde los CSV) y verificación
python -m benchmarks.generate_report_tables
python -m benchmarks.generate_report_tables --check

# Pruebas
python -m pytest -q
```

El benchmark espacial usa PostGIS como baseline (GiST sobre `geometry(Point,4326)`); si PostGIS no está instalado, puede usar la extensión `earthdistance` con `-pg-mode earthdistance`. El benchmark deja constancia en el JSON de resultados del backend usado (`"gist_backend"` y `"postgis_available"`) y de la versión del servidor, para que no haya duda de qué baseline se midió.